

INFORME DE COMPLEJIDAD Y RENDIMIENTO

1. Introducción

En este informe se analiza el comportamiento de diferentes partes del proyecto teniendo en cuenta la cantidad de datos que deben procesar. La idea principal es identificar qué tan rápido crece el tiempo de ejecución cuando aumenta el tamaño de la entrada.

2. Resumen de resultados

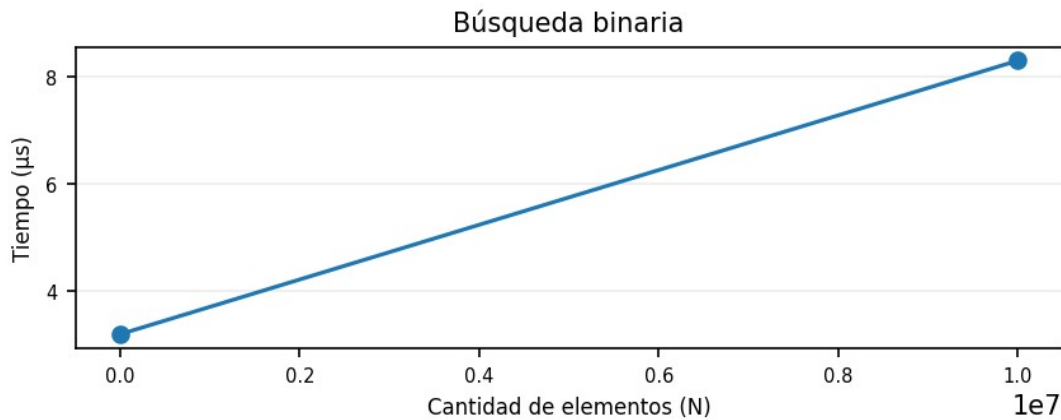
Elemento	Complejidad	Comportamiento observado
Arreglo dinámico	Acceso $O(1)$. Agregar $O(1)$ amortizado. Insertar/eliminar $O(N)$.	Agregar se mantiene casi constante; insertar y eliminar aumentan con el tamaño.
Búsqueda binaria	$O(\log N)$. Mejor caso $O(1)$.	Pasó de 3,2 μ s a 8,3 μ s mientras N aumentó de 1.000 a 10.000.000.
Carga y guardado	$O(N)$.	El crecimiento esperado es lineal; el tiempo también depende del sistema de archivos.
Insertion Sort	Peor/promedio $O(N^2)$. Mejor $O(N)$.	Los casos no favorables aumentan rápidamente, mientras el mejor caso es mucho menor.
Merge Sort	$O(N \log N)$.	En $N=80.000$, la versión 2 tardó aproximadamente la mitad que la versión 1.
Cursor	$O(1)$ por movimiento.	Los cuatro movimientos estuvieron alrededor de 0,23–0,27 μ s.
Pruebas unitarias	Depende del componente probado.	Insertion ≈ 230 ms; Merge ≈ 13 ms; pruebas del cursor < 1 ms.

3. Arreglo dinámico, búsqueda binaria y manejo de archivos

Arreglo dinámico: las operaciones de acceso mediante índice tienen complejidad $O(1)$, porque no es necesario recorrer los elementos anteriores. La operación agregar tiene un costo $O(1)$ amortizado, aunque cuando el arreglo necesita crecer se realiza una redimensión que cuesta $O(N)$. En cambio, insertar o eliminar elementos puede requerir desplazar varios valores y por eso puede llegar a $O(N)$.

En las mediciones disponibles, agregar presentó tiempos cercanos a 0,63–0,77 μ s para los tamaños analizados. Para insertar y eliminar se reportaron aproximadamente 1,8 ms con $N=10.000$ y 15 ms con $N=80.000$. Esto muestra la diferencia entre una operación que normalmente no necesita recorrer el arreglo y otra que puede requerir mover muchos elementos.

Búsqueda binaria: este método trabaja sobre datos ordenados y en cada paso reduce aproximadamente a la mitad el espacio donde puede estar el elemento buscado. Por esta razón su complejidad es $O(\log N)$, mientras que el mejor caso es $O(1)$ cuando el elemento se encuentra inmediatamente.

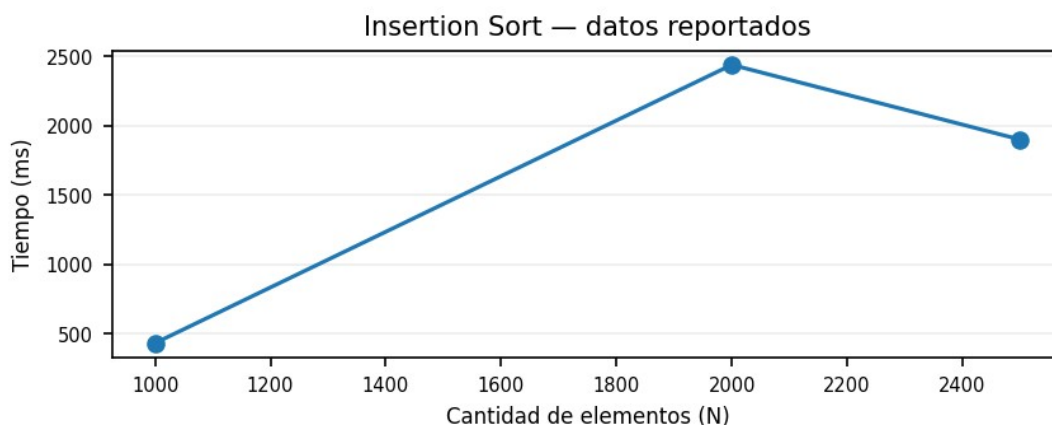


Carga y guardado de archivos: leer o escribir N líneas implica procesar una cantidad de información que crece con N, por lo que la complejidad esperada es $O(N)$. En la práctica, el tiempo puede verse afectado .

4. Ordenamiento de datos

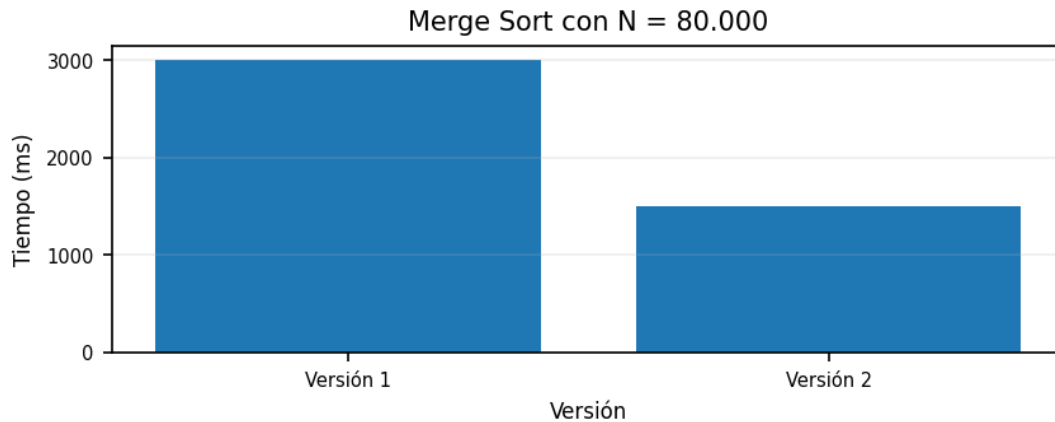
Insertion Sort: construye el arreglo ordenado tomando los elementos uno por uno. Cuando los elementos están desordenados, puede ser necesario realizar muchas comparaciones y desplazamientos. Por eso su complejidad en el peor caso y en el caso promedio es $O(N^2)$. Cuando los datos ya están ordenados, el comportamiento mejora a $O(N)$.

Entre los datos reportados aparecen aproximadamente 430 ms para $N=1.000$ y 2.440 ms para $N=2.000$ en un caso no favorable. También se registró un promedio cercano a 1.900 ms con $N=2.500$ y un mejor caso de aproximadamente 1,2 ms para ese mismo tamaño. Los valores muestran que el orden inicial de los datos tiene un efecto importante en el tiempo.



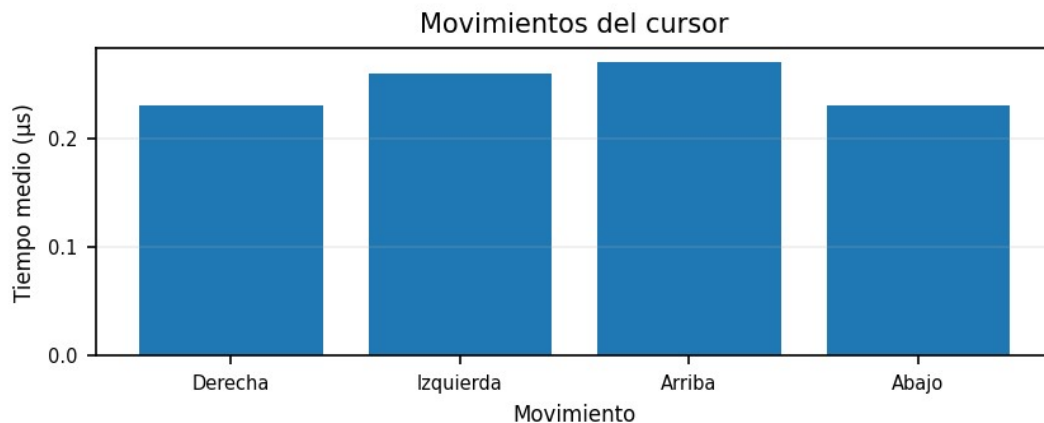
Merge Sort. Las dos versiones analizadas tienen complejidad $O(N \log N)$. La diferencia aparece principalmente en la forma en que manejan la memoria durante el proceso de ordenamiento.

Para $N=80.000$, la primera versión fue reportada alrededor de 3.000 ms y la segunda alrededor de 1.500 ms. La primera crea varios subarreglos durante la recursión, lo que implica más creación de objetos y trabajo adicional. La segunda reutiliza el arreglo original y necesita menos estructuras temporales. Por eso, aunque ambas tienen la misma Big-O, sus tiempos reales pueden ser diferentes.



5. Posición del cursor y pruebas

Posición del cursor: Las operaciones para mover el cursor a la derecha, izquierda, arriba o abajo tienen complejidad $O(1)$. Esto se debe a que cada movimiento realiza una cantidad fija de operaciones y no depende de que el archivo tenga más o menos contenido.



Pruebas unitarias: su tiempo depende de qué componente se esté ejecutando. En los resultados disponibles, una prueba relacionada con Insertion Sort sobre 1.000 líneas tardó aproximadamente 230 ms, mientras que una prueba de Merge Sort sobre 1.000 líneas tardó aproximadamente 13 ms. Las pruebas relacionadas con el cursor estuvieron por debajo de 1 ms.

Durante la revisión también se encontraron algunos detalles que debían corregirse en el código de las pruebas: la ubicación de un return dentro de un ciclo, el nombre correcto de ValueError, el uso de ascii_lowercase y la forma de acceder al atributo de columna del cursor. Estos cambios son importantes porque una prueba debe coincidir con la forma en que está implementado el programa.

6. Análisis general y conclusiones

Al comparar la parte teórica con los resultados prácticos, se observa que en general el comportamiento sigue lo esperado. Las operaciones $O(1)$, como los movimientos del cursor, presentan tiempos prácticamente constantes. La búsqueda binaria también muestra un crecimiento pequeño a pesar del aumento considerable de la cantidad de datos, lo cual corresponde a $O(\log N)$.

En el arreglo dinámico se observa que no todas las operaciones tienen el mismo costo. Agregar elementos es rápido en promedio, pero insertar y eliminar pueden necesitar desplazamientos y por eso aumentan cuando el tamaño del arreglo es mayor.

En los algoritmos de ordenamiento también se aprecia una diferencia clara. Insertion Sort puede aumentar rápidamente en los casos menos favorables debido a su $O(N^2)$, mientras que Merge Sort mantiene $O(N \log N)$. Además, la comparación entre las dos versiones de Merge Sort demuestra que la misma complejidad teórica no significa que dos implementaciones tengan exactamente el mismo tiempo de ejecución.